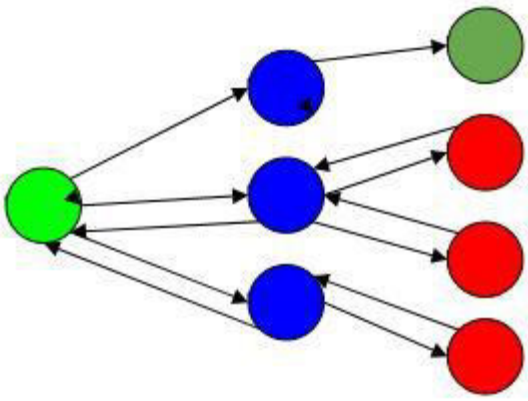
BACKTRACKING

- Backtracking is an algorithmic-technique for solving problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point of time
- **Backtracking** is a general algorithm for finding all (or some) solutions to some computational problems, notably constraint satisfaction problems, that incrementally builds candidates to the solutions, and abandons a candidate ("backtracks") as soon as it determines that the candidate cannot possibly be completed to a valid solution.
- The backtracking algorithm traverses this search tree recursively, from the root down, in depth-first order.
- At each node *c*, the algorithm checks whether *c* can be completed to a valid solution.
- If it cannot, the whole sub-tree rooted at *c* is skipped (*pruned*).
- Otherwise, the algorithm
    (1) Checks whether *c* itself is a valid solution, and if so reports it to the user; and
    (2) Recursively enumerates all sub-trees of *c*. The two tests and the children of each node are defined by user-given procedures.
- It removes the solution that doesn't give rise to the solution of the problem based on the constraints given to solve the problem.

Backtracking algorithm is applied to some specific types of problems,

- ✓ Decision problem used to find a feasible solution of the problem.
- ✓ Optimisation problem used to find the best solution that can be applied.
- ✓ Enumeration problem used to find the set of all feasible solutions of the problem.
    - In backtracking problem, the algorithm tries to find a sequence path to the solution which has some small checkpoints from where the problem can backtrack if no feasible solution is found for the problem.

EXAMPLE



- Green is the start point, blue is the intermediate point, red are points with no feasible solution, dark green is end solution.
- Here, when the algorithm propagates to an end to check if it is a solution or not, if it is then returns the solution otherwise backtracks to the point one step behind it to find track to the next point to find solution.

CONTROL ABSTRACTION

Procedure Backtrack (n)

{

K=1

While **K > 0** do

if there remained an untried X (K) such that

X (K) Є T (X (1)…X (k-1)) and B_k X (1)… X (K)) = true then

if (X (1)… X (k)) is a path to an answer node then

print (X (1),…X (k))

end if

K=K+1 // consider next set //

else K=K-1 // backtrack to previous set

end if

repeat

end Backtrack

}